📘 MÓDULO 2: GITHUB ACTIONS, WORKFLOWS (YAML) Y MAKEFILE

---

1. UBICACIÓN Y ESTRUCTURA OBLIGATORIA

Los flujos de trabajo automatizados (Workflows) deben escribirse obligatoriamente en formato **YAML** (.yml o .yaml) y guardarse en una ruta muy estricta dentro de tu proyecto.

- **Ruta exacta:** .github/workflows/nombre-workflow.yml
- *¡Si no usas exactamente esta ruta y estructura de carpetas, GitHub Actions ignorará por completo tus archivos!*

2. ESTRUCTURA BÁSICA DE UN WORKFLOW (YAML)

Aquí tienes una estructura completa y comentada que te sirve perfectamente para el examen:

```
name: Examen Workflow CI/CD

# ⚡ 1. Disparador (Cuándo se ejecuta)
on:
  push:
    branches: [ main, master ]

# 🏗️ 2. Trabajos (Jobs) que va a realizar el robot
jobs:
  construir-y-probar:
    # 🖥️ Entorno virtual (ej. ubuntu-latest o windows-latest)
```

```
    runs-on: ubuntu-latest

    # 🚶 Pasos secuenciales del trabajo
    steps:
      # Paso 1: Descargar el código del repositorio en la máquina virtual
      - name: Checkout del repositorio
        uses: actions/checkout@v3

      # Paso 2: Ejecutar comandos de consola directos (Run)
      - name: Imprimir mensaje de éxito
        run: echo "¡El workflow de examen se ha ejecutado correctamente!"
```

⚠️ Diferencias clave entre uses y run (¡Pregunta típica de examen!):

- **uses:** Se usa exclusivamente para invocar **Actions prediseñadas** creadas por GitHub o por la comunidad (ej. actions/checkout@v3).
- **run:** Se usa para ejecutar **comandos de terminal o scripts de shell** directamente (ej. echo, npm test, o llamadas a un make).

3. ¿CÓMO ADAPTARLO PARA WINDOWS O UBUNTU?

En la línea runs-on:, puedes cambiar el sistema operativo de la máquina virtual (*runner*) según lo que te pida el profesor:

- **runs-on: ubuntu-latest ➔** Ejecuta el workflow en una máquina virtual con Linux (la más común y rápida).
- **runs-on: windows-latest ➔** Ejecuta el workflow en una máquina virtual con Windows (ideal si tu aplicación o comandos requieren PowerShell o entorno Windows).

4. ¿QUÉ ES UN MAKEFILE Y CÓMO SE USA CON GIT / ACTIONS?

📌 ¿Qué es un Makefile?

Es un archivo de automatización local (un script plano llamado exactamente Makefile sin ninguna extensión) que contiene una lista de reglas e instrucciones cortas para compilar código, ejecutar pruebas o empaquetar proyectos ejecutando una sola palabra clave (make).

🛠️ Estructura básica de un Makefile:

```
# Ejemplo de Makefile básico para examen
saludar:
	echo "Hola, esto se ejecuta con make"

compilar:
	gcc main.c -o programa
```

(Nota de examen: Las líneas de comandos dentro de un Makefile van tabuladas obligatoriamente, no con espacios).

🚀 ¿Cómo se integra un Makefile con GitHub Actions?

En tu archivo YAML de workflow, puedes invocar las órdenes de tu Makefile utilizando la directiva run: dentro de los steps:

```
  steps:
    - name: Checkout del código
      uses: actions/checkout@v3
```

```
    - name: Ejecutar la tarea de compilación del Makefile
      run: make compilar
```

De esta forma, cuando haces push, el robot de GitHub descarga tu código y ejecuta automáticamente el make que tenías preparado en tu proyecto.